Lava Tracker SDK — Developer Guide

> **Status:** SP-3a documentation-polish revision (2026-05-01, Lava-Android-1.2.0-1020). The 7-step recipe in §2 remains the load-bearing structural acceptance criterion; sections 1, 3, 4 are the conceptual frame; sections 5–7 are mechanical compliance gates added in the polish pass and bind every PR that adds a tracker.
>
> **Validation:** Each step in §2 has been paper-traced through the existing `:core:tracker:rutor` module (added in SP-3a Phase 3). If you can answer "yes" to every step against `core/tracker/rutor/`, your new tracker module is structurally compliant. Sections 5–7 are validated against both `:core:tracker:rutracker` and `:core:tracker:rutor` — if a new tracker can satisfy them as completely as those two do, the SDK contract is preserved.

---

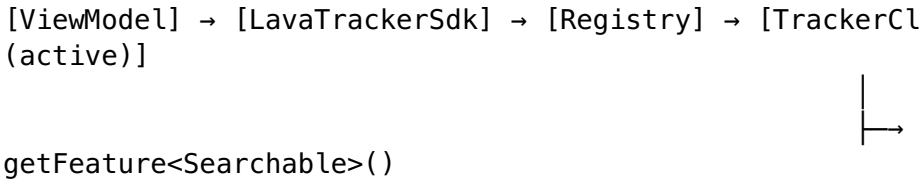
1. SDK Architecture Overview

The Lava Tracker SDK lives in two layers:

1. **Generic primitives** — mounted at `submodules/tracker_sdk/` (the `vasic-digital/Tracker-SDK` submodule). These are Lava-agnostic building blocks: `MirrorUrl`, `TrackerDescriptor`-shape, `Mirror` health-state machine, in-memory registry, mirror-config store, and test scaffolding (clock, dispatcher, fixture loader). The submodule is **frozen by default**: updating the pin is a deliberate PR.

2. **Lava-domain wrappers + per-tracker plugins** — under `core/tracker/` in the Lava monorepo. `core/tracker/api/` exposes the seven feature interfaces (`Searchable`, `Browsable`, `Topic`, `Comments`, `Favorites`, `Authenticatable`, `Downloadable`) and the `TrackerCapability` enum. `core/tracker/{rutracker,rutor}/` are the two shipped per-tracker plugins. `core/tracker/client/` hosts `LavaTrackerSdk`, the Hilt-injected orchestrator that ViewModels consume.

A feature ViewModel never imports a per-tracker module directly. It talks to `LavaTrackerSdk`, which routes the call through the active tracker's `TrackerClient.getFeature<T>()`. Capability declared in the descriptor ⇒ feature interface returned ⇒ method works (constitutional clause 6.E, Capability Honesty).



```
                                                              ├─→
getFeature<Authenticatable>()
                                                              └─→
getFeature<Downloadable>()
```

When all mirrors of the active tracker fail, `LavaTrackerSdk` emits `CrossTrackerFallbackProposed` (Phase 4); the UI presents a modal offering the alternative tracker. There is no silent fallback.

---

2. Adding a New Tracker (7-step recipe)

Follow these steps in order. Each step is testable against the existing `:core:tracker:rutor` module — when in doubt, paper-trace there first.

Step 1 — Create the Gradle module

```
mkdir -p core/tracker/<trackerId>/src/{main,test}/kotlin
mkdir -p core/tracker/<trackerId>/src/test/resources/fixtures/
         <trackerId>
```

Create `core/tracker/<trackerId>/build.gradle.kts`:

```
plugins {
    id("lava.kotlin.tracker.module")
}
```

That convention plugin pre-wires `:core:tracker:api`, `lava.sdk:api`, `lava.sdk:mirror`, Jsoup, OkHttp, kotlinx-coroutines, kotlinx-serialization, JUnit4, mockk, kotlinx-coroutines-test, `:core:tracker:testing`, and `lava.sdk:testing`. **Do not duplicate config**; if you find yourself adding dependencies module-by-module, extend the convention plugin instead.

Add the module to `settings.gradle.kts`:

```
include(":core:tracker:<trackerId>")
```

Step 2 — Define the `TrackerDescriptor`

In `core/tracker/<trackerId>/src/main/kotlin/lava/tracker/<trackerId>/<TrackerId>Descriptor.kt`:

```
object <TrackerId>Descriptor : TrackerDescriptor {
    override val id = "<trackerId>"
    override val displayName = "<Display Name>"
    override val capabilities: Set<TrackerCapability> = setOf(
        TrackerCapability.SEARCH,
        TrackerCapability.TOPIC,
        TrackerCapability.DOWNLOAD,
        // declare ONLY the capabilities you will actually wire up
        // (clause 6.E Capability Honesty: declared ⇒ getFeature
         non-null)
```

```
    )
    override val defaultMirrors = listOf(
        MirrorUrl(host = "<primary-host>", priority = 100),
        MirrorUrl(host = "<secondary-host>", priority = 50),
    )
    override val authRequired = false  // or true for RuTracker-
        style auth
}
```

Step 3 — Implement the per-feature interfaces

For each capability declared in step 2, write a class implementing the matching `core:tracker:api` interface. Classes live in `core/tracker/<trackerId>/src/main/kotlin/lava/tracker/<trackerId>/`:

- `<TrackerId>Search.kt` → `Searchable`
- `<TrackerId>Topic.kt` → `Topic`
- `<TrackerId>Download.kt` → `Downloadable`
- `<TrackerId>Authenticator.kt` → `Authenticatable` (if applicable)
- (etc.)

Each implementation MUST use OkHttp + Jsoup against the live tracker HTML. **No HTTP-mocking inside the production class** — the `OkHttpClient` is constructor-injected so a test can swap in `MockWebServer` (clause 6.E + Seventh Law clause 4: mock at the boundary, not inside the SUT).

Step 4 — Wire up the `TrackerClient`

In `core/tracker/<trackerId>/src/main/kotlin/lava/tracker/<trackerId>/<TrackerId>Client.kt`:

```
class <TrackerId>Client @Inject constructor(
    private val search: <TrackerId>Search,
    private val topic: <TrackerId>Topic,
    private val download: <TrackerId>Download,
) : TrackerClient {
    override val descriptor = <TrackerId>Descriptor

    override fun <T : Any> getFeature(klass: KClass<T>): T? =
        when (klass) {
        Searchable::class    -> if (TrackerCapability.SEARCH   in
        descriptor.capabilities) search   as T else null
        Topic::class         -> if (TrackerCapability.TOPIC    in
        descriptor.capabilities) topic    as T else null
        Downloadable::class  -> if (TrackerCapability.DOWNLOAD in
        descriptor.capabilities) download as T else null
        else -> null
    }
}
```

The capability check inside `getFeature` is the runtime enforcement of clause 6.E. Drop a capability from the descriptor → `getFeature` returns null → call site MUST handle null (no silent stub).

Step 5 — Register the `TrackerClientFactory`

In `core/tracker/client/src/main/kotlin/lava/tracker/client/registry/` add:

```
@Module
@InstallIn(SingletonComponent::class)
object <TrackerId>RegistrationModule {
    @Provides
    @IntoSet
    fun provide<TrackerId>Factory(
        impl: <TrackerId>Client,
    ): TrackerClientFactory = TrackerClientFactory(
        descriptor = <TrackerId>Descriptor,
        instantiate = { impl },
    )
}
```

The `@IntoSet` multi-binding picks every factory up at app start; the registry exposes them by `descriptor.id`. No reflection, no service loader, no manual bootstrapping in `Application.onCreate`.

Step 6 — Drop fixtures + write parser tests

Save sample HTML pages from the live tracker (one per parser scope) at:

```
core/tracker/<trackerId>/src/test/resources/fixtures/<trackerId>/
  search/<query>-<YYYY-MM-DD>.html
  topic/<topicId>-<YYYY-MM-DD>.html
  ...
```

Fixture filenames MUST embed the date (`scripts/check-fixture-freshness.sh` warns at >30 days, blocks at >60 days). Each parser scope MUST have at least 5 fixtures spanning real-world variations.

Write parser tests under `src/test/kotlin/`:

```
class <TrackerId>SearchParserTest {
    @Test fun `parses search results from 2026-04 fixture`() {
        val html = readFixture("search/ubuntu-2026-04-15.html")
        val results = <TrackerId>SearchParser.parse(html)
        assertEquals(50, results.size)
        assertEquals("ubuntu-22.04.4-desktop-amd64.iso",
         results[0].title)
        assertTrue(results[0].sizeBytes > 0)
        assertTrue(results[0].seeders >= 0)
    }
}
```

Every test commit MUST carry a `Bluff-Audit:` stamp in the commit body (Seventh Law clause 1, mechanical pre-push hook enforcement).

Step 7 — Add a Challenge Test in `:app`

In `app/src/androidTest/kotlin/lava/app/challenges/`, add a Compose UI test that exercises the full stack against your new tracker. Mirror the shape of `C1_AppLaunchAndTrackerSelectionTest.kt` (added in SP-3a Phase 5). The Challenge Test:

1. Drives `MainActivity` to the search screen.
2. Switches the active tracker to your new tracker via Settings → Trackers.
3. Performs a real search.
4. Asserts ≥1 result row renders with parseable size + seeders text.

Falsifiability rehearsal: temporarily mutate the tracker's `getFeature` to return null for `Searchable` → re-run → confirm the UI shows the "not supported" path → revert. Capture the rehearsal in `.lava-ci-evidence/sp3a-challenges/<TestName>-<sha>.json` per Sixth Law clause 2.

---

3. Mirror Configuration

Mirror sources, in priority order:

1. **Bundled defaults** — `core/tracker/client/src/main/assets/mirrors.json`, merged at app start by `MirrorConfigLoader`.
2. **User-added custom mirrors** — persisted in Room (`tracker_mirror_user` table), exposed by `UserMirrorRepository`.
3. **Health-tracked active subset** — `MirrorHealthRepository` writes `tracker_mirror_health` rows on every probe; the `MirrorHealthCheckWorker` runs every 15 minutes and writes `HEALTHY` / `DEGRADED` / `UNHEALTHY` states.

When all mirrors of the active tracker hit `UNHEALTHY`, `LavaTrackerSdk` emits a `CrossTrackerFallbackProposed` outcome. The UI shows `CrossTrackerFallbackModal`; user accept → re-issues the call on the alternative tracker; user dismiss → explicit failure UI (snackbar). No silent fallback.

To add a tracker's default mirrors, edit `mirrors.json`:

```
{
  "<trackerId>": [
    { "host": "<primary>", "priority": 100 },
    { "host": "<secondary>", "priority": 50 }
  ]
}
```

`MirrorConfigLoader` validates this on app start; an entry without a matching `TrackerDescriptor` is logged as a config error and ignored.

---

4. Testing Requirements

Every tracker module MUST satisfy:

Requirement	Mechanism
Real-stack parser tests	Fixtures under `src/test/resources/fixtures/<id>/`, one test per parser scope, ≥5 fixtures each. Pre-push hook rejects new test commits without a `Bluff-Audit:` stamp.
Capability honesty	Constitutional clause 6.E: every capability declared in the descriptor MUST resolve to a non-null `getFeature<T>()`. The `:core:tracker:registry` test enumerates descriptors + capabilities and asserts non-null.
Cross-tracker parity	Where two trackers expose the same capability (e.g. both have `SEARCH`), at least one shared-shape test asserts the result-set DTO is identical (modulo tracker id).
Real-tracker integration test	Gated by `-PrealTrackers=true`. Hits the live tracker. Run before tagging; results recorded in `.lava-ci-evidence/sp3a-rutor/` (or equivalent for new trackers).
Real-device Challenge Test	One Compose UI test per user-visible scenario in `app/src/androidTest/kotlin/lava/app/challenges/`. Falsifiability rehearsal captured in `.lava-ci-evidence/sp3a-challenges/`. Operator real-device attestation required before the next release tag.

Coverage exemptions for individual lines go in `docs/superpowers/specs/<spec>-coverage-exemptions.md` per clause 6.D. Blanket waivers are forbidden.

---

5. Reference: existing trackers

Tracker	Module	Capabilities	Auth
RuTracker	`:core:tracker:rutracker`	SEARCH + BROWSE + TOPIC + COMMENTS + FAVORITES + AUTH + DOWNLOAD	Required (login)
RuTor	`:core:tracker:rutor`	SEARCH + TOPIC + DOWNLOAD	Anonymous (decision 7b-ii)

When adding the third tracker, follow this guide; if any step doesn't match the existing modules, that is the first thing to clarify in the PR description.

---

5. Real-Stack Testing Requirements (Seventh Law clauses)

The Sixth Law mandates real-user verification; the Seventh Law (added 2026-04-30) mandates the **mechanical** way that verification is enforced for tracker plugins. A new tracker MUST satisfy each of the seven clauses below before the next release tag.

5.1 — Bluff-Audit stamp on every test commit (clause 1)

Every commit whose diff touches a `*Test.kt` file MUST carry a `Bluff-Audit:` block in the commit message body. The pre-push hook at `.githooks/pre-push` rejects pushes without it. The required shape:

```
Bluff-Audit:
  Test:      lava.tracker.<id>.feature.<TestClassName>
  Mutation:  <one sentence describing the deliberate production-
code break>
  Observed:  <verbatim assertion message produced by the mutated
test run>
  Reverted:  <commit SHA or "yes (worktree)" confirming the
mutation was undone>
```

A stamp that says "Mutation: none / Observed: nothing / Reverted: nothing" is an explicit Seventh Law violation and blocks the push.

5.2 — Real-stack verification gate (clause 2)

Each user-visible tracker capability MUST have a real-stack test:

Capability surface	Real-stack gate
Tracker network parsing	Live HTTP fixture run via `./gradlew :core:tracker:<id>:integrationTest -PrealTrackers=true`
Mirror health probe	Live mirror HEAD via the same `-PrealTrackers=true` flag
Cross-tracker fallback UX	`:app:connectedDebugAndroidTest --tests "lava.app.challenges.Challenge0[78]*"` on a real device

If a capability cannot be real-stack-tested, it is documented as "not user-reachable" in the coverage exemption ledger and flagged as a deferred gap in the PR description — never silently shipped.

5.3 — Pre-tag real-device attestation (clause 3)

Every release tag carrying tracker code MUST be preceded by:

```
.lava-ci-evidence/Lava-Android-<vname>-<vcode>/real-device-
verification.md
```

with `status: VERIFIED` plus, for each Challenge Test C1–C8 that exercises the new tracker, a per-test attestation file under `.lava-ci-evidence/Lava-Android-<vname>-<vcode>/challenges/C<n>.json` also at `status: VERIFIED`. `scripts/tag.sh` refuses to tag without all of these in place.

5.4 — Forbidden test patterns (clause 4)

The pre-push hook rejects:

- `mockk<XxxClass>(...)` inside `XxxClassTest.kt` (mocking the SUT).
- Tests whose primary assertion is `verify { mock.foo() }`.
- `@Ignore`'d tests without a tracking issue ID in a comment.
- Tests that build the SUT but never invoke its public methods.
- Acceptance gates whose assertion is `BUILD SUCCESSFUL` (literal or paraphrased).

If your test seems to require one of these, refactor the SUT for testability — the test is telling you the design is wrong.

5.5 — Recurring bluff hunt (clause 5)

Every 2–4 weeks of active work, an end-of-phase bluff hunt picks 5 random `*Test.kt` files, deliberately mutates the production code each test claims to cover, and confirms the test fails. Output is appended to `.lava-ci-evidence/bluff-hunt/<date>.json`. New tracker modules become eligible for the random sample on first commit; expect their parser tests to be hunted by the third or fourth phase wrap.

5.6 — Bluff discovery protocol (clause 6)

If a real user files a bug whose tests were green: declare a Seventh Law incident, write a regression test that fails before the fix, record the bluff diagnosis under `.lava-ci-evidence/sixth-law-incidents/<date>.json`, and add the bluff pattern to the forbidden-pattern list at the top of this section.

5.7 — Inheritance (clause 7)

This section MUST be linked or copied verbatim into every per-tracker module's `README.md` (and into any future `vasic-digital` extension submodule). Forking a non-compliant external tracker plugin is forbidden — re-implement under `vasic-digital/` instead.

> See also: root `CLAUDE.md` Seventh Law (canonical text), root `AGENTS.md` Seventh Law mirror, `core/CLAUDE.md` scoped clause for per-tracker modules.

---

6. Falsifiability Rehearsal Protocol (worked example)

The mechanical Bluff-Audit stamp (§5.1) requires a `Mutation` line. The mutation MUST be **a real edit to the production code that the test claims to cover**, not a tweak to the test itself. The pattern:

1. **Identify the load-bearing assertion.** Read the test, identify the one assertion that, if it fails, means a real user is broken.
2. **Locate the production code that must be correct for that assertion to pass.** Resist the urge to mutate the test fixture — you are auditing the production code, not the test scaffold.
3. **Apply a minimal mutation.** Single-character, single-branch, single-method-rename. Smaller is better; the goal is "did this test actually exercise that line?"
4. **Run the test once with the mutation in place.** Capture the verbatim assertion failure message, including the values it reported.
5. **Revert the mutation.** Run again, confirm pass.
6. **Record the four lines** in the commit body Bluff-Audit stamp.

Worked example — `RuTorSearchParserTest`

The test in `core/tracker/rutor/src/test/kotlin/lava/tracker/rutor/feature/RuTorSearchTest.kt` asserts that the parser extracts a non-empty result list with at least one row whose `seeders >= 0` from the bundled fixture `fixtures/rutor/search/ubuntu-2026-04-15.html`.

Mutation candidate (good): in `core/tracker/rutor/src/main/kotlin/lava/tracker/rutor/parser/SearchParser.kt`, change `select("td.s")` (the seeders column selector) to `select("td.q")` (a non-existent class). Re-run the test — the production code now extracts no rows; the test fails with `expected: <non-empty list> but was: <[]>` (or whatever shape the production class returns on parser failure). Revert. Re-run. Pass.

Mutation candidate (bad — DO NOT USE): changing the fixture HTML to remove all `<tr>` rows. The mutation is on the *test fixture*, not the *production code*; this proves only that the parser handles empty input, not that it correctly handles real input.

Mutation candidate (bad — DO NOT USE): changing the test assertion from `assertTrue(results.isNotEmpty())` to `assertFalse(results.isNotEmpty())`. This is a tautological mutation of the test itself; it proves nothing.

What the Bluff-Audit stamp looks like

```
sp3a-3.7-rutor-search: real-stack parser test on
ubuntu-2026-04-15.html

Adds RuTorSearchParserTest, exercising the production SearchParser
against a saved live response.

Bluff-Audit:
```

```
  Test:
lava.tracker.rutor.feature.RuTorSearchTest.parses_search_results_fr
  Mutation:  Renamed `select("td.s")` to `select("td.q")` in
SearchParser.kt
             so the seeders column resolves to an empty NodeSet.
  Observed:  expected: <non-empty list> but was: <[]> at line 42
of RuTorSearchTest.kt
  Reverted:  yes (commit 8a32e1b shows the un-mutated source); re-
running
             the test after revert produces 50 rows with seeders
>= 0.

Co-Authored-By: ...
```

When the rehearsal is impractical

If the mutation rehearsal cannot be completed for legitimate reasons (e.g. the test runs in an environment the agent cannot access on a real device), record `Reverted: pending operator` and add an operator-checklist entry to `.lava-ci-evidence/Lava-Android-<v>/real-device-verification.md`. The tag gate then refuses to operate until the operator completes the rehearsal and updates the stamp by amendment-on-merge.

---

7. Mirror Configuration Spec (`mirrors.json`)

`core/tracker/client/src/main/assets/mirrors.json` is the bundled default-mirror set merged at app start by `MirrorConfigLoader`. It is the **only** way to ship a default mirror with the app — runtime additions live in the `tracker_mirror_user` Room table and never touch this file.

JSON schema

```json
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "additionalProperties": false,
  "required": ["version", "trackers"],
  "properties": {
    "version": {
      "type": "integer",
      "minimum": 1,
      "description": "Bumped when a breaking shape change lands;
        MirrorConfigLoader rejects unknown versions."
    },
    "generatedAt": {
      "type": "string",
      "format": "date-time",
      "description": "ISO-8601 timestamp the file was last
        regenerated."
    },
```

```
    "trackers": {
      "type": "object",
      "patternProperties": {
        "^[a-z][a-z0-9_]{1,31}$": {
          "type": "array",
          "minItems": 1,
          "items": {
            "type": "object",
            "additionalProperties": false,
            "required": ["url", "priority", "protocol"],
            "properties": {
              "url":      { "type": "string", "format": "uri" },
              "priority": { "type": "integer", "minimum": 0,
        "maximum": 1000 },
              "protocol": { "type": "string", "enum": ["HTTPS",
        "HTTP"] },
              "isPrimary": { "type": "boolean", "default":
        false },
              "region":    { "type": "string", "description":
        "Free-form region tag, e.g. 'eu', 'ipv6-only'." }
            }
          }
        },
        "additionalProperties": false
    }
  }
}
```

Worked example

```
{
  "version": 1,
  "generatedAt": "2026-04-30T19:20:00Z",
  "trackers": {
    "rutracker": [
      { "url": "https://rutracker.org", "isPrimary": true,
        "priority": 0,  "protocol": "HTTPS" },
      { "url": "https://rutracker.net", "priority": 1,
        "protocol": "HTTPS" },
      { "url": "https://rutracker.cr",  "priority": 2,
        "protocol": "HTTPS" }
    ],
    "rutor": [
      { "url": "https://rutor.info", "isPrimary": true,
        "priority": 0, "protocol": "HTTPS" },
      { "url": "https://rutor.is",   "priority": 1, "protocol":
        "HTTPS" },
      { "url": "http://6tor.org",    "priority": 4, "protocol":
        "HTTP", "region": "ipv6-only" }
    ]
  }
}
```

Validation rules (enforced by `MirrorConfigLoader` at runtime)

1. **Tracker key MUST match a registered `TrackerDescriptor.trackerId`.** An entry whose key has no descriptor is logged as a config error and ignored — it is never silently downgraded to a fallback default.
2. **`isPrimary: true` MUST appear at most once per tracker.** Two primaries triggers a config error and the loader keeps the first-encountered one.
3. **`priority` is unique-by-convention** (lower wins). Duplicates are accepted but the order between equally-prioritised mirrors is undefined.
4. **`protocol` MUST match `url` scheme.** A `protocol: HTTPS` entry with an `http://` URL is rejected.
5. **`region` is purely informational.** It is not used for routing logic in 1.2.0; reserved for future region-aware health probes.

When you regenerate `mirrors.json`

- Bump `generatedAt`. (`version` is bumped only when the schema itself changes.)
- Run `./gradlew :core:tracker:client:test` — the loader has parser tests that catch shape regressions.
- Add a Bluff-Audit stamp to the commit per §5.1 — even a JSON file edit that adjusts default mirrors is a behavior change worthy of rehearsal (e.g. mutate one URL to a non-resolving host, run the health-probe test, observe the UNHEALTHY emit, revert).

Forbidden in `mirrors.json`

- Credentials, tokens, captcha cookies — these are user-scoped and belong in encrypted preferences, never in a bundled asset.
- LAN-only addresses (`192.168.*`, `10.*`, `.local`) — those are discovery-time concerns owned by `LocalNetworkDiscoveryService` in `:core:data`.
- Onion / I2P / proxy hostnames — out of scope for 1.2.0.

7.1 HTTP file download (`HttpDownloadableTracker` / `HTTP_DOWNLOAD`)

Not every provider serves `.torrent` files. Internet Archive (`archiveorg`) and Project Gutenberg (`gutenberg`) are digital libraries that serve their artifact (EPUB / plain-text / HTML e-books, raw media) over plain HTTP(S). Routing those bytes through `DownloadableTracker.downloadTorrentFile` would be a §6.E bluff: the consumer expects a bencoded `.torrent`, not an e-book.

The honest home for them is a distinct capability + feature interface:

```
interface HttpDownloadableTracker : TrackerFeature {
    /** Bytes + resolved source URL + suggested filename. Throws
        on non-2xx / empty. */
    suspend fun downloadHttpFile(id: String): HttpDownloadResult
}
```

```
data class HttpDownloadResult(val bytes: ByteArray, val
        sourceUrl: String, val fileName: String)
```

A provider that serves files over HTTP declares `TrackerCapability.HTTP_DOWNLOAD` in its descriptor and resolves `HttpDownloadableTracker` (NOT `DownloadableTracker`) from `getFeature`. The §6.E gate (`CapabilityHonestyContractTest`, forward + reverse) enforces both directions: `HTTP_DOWNLOAD` declared ⇒ `getFeature(HttpDownloadableTracker)` non-null, and `HTTP_DOWNLOAD` undeclared ⇒ it stays null. `archiveorg` + `gutenberg` are the reference implementations.

> **SDK-layer note (LVA-044):** the capability + interface + per-provider real-stack tests are wired at the SDK layer. End-to-end UI wiring (a download surface that calls `downloadHttpFile` and writes the artifact to disk for the user) is still OWED — the current topic/download screen only handles `.torrent`/magnet via `DownloadableTracker`. Until that lands, `HTTP_DOWNLOAD` is honestly SDK-reachable, not yet end-user-reachable.

8. The magnet-cache pattern (§6.E Capability Honesty)

`DownloadableTracker` declares two methods:

```
interface DownloadableTracker : TrackerFeature {
    suspend fun downloadTorrentFile(id: String): ByteArray
    /** Returns null if the magnet URI is not synchronously
        available without an HTTP fetch. */
    fun getMagnetLink(id: String): String?
}
```

`getMagnetLink` is the **only non-suspend** method on any feature interface — so it **cannot** make an HTTP call. Its contract is: return the magnet *only if synchronously available*, else `null`. Historically the no-magnet providers just `return null` unconditionally — but that is a §6.E **bluff** when the descriptor declares `MAGNET_LINK`: the capability is advertised, yet the feature never produces a magnet.

The fix that `kinozal`, `nnmclub`, and `rutor` use — and that **every new provider declaring `MAGNET_LINK` MUST follow** — is a per-provider **magnet cache**: a `@Singleton` in-memory map that the *suspend* topic/search path populates from genuinely-parsed magnets, read back by the *non-suspend* `getMagnetLink`.

8.1 The cache

```
// core/tracker/<id>/.../<dir>/<TrackerId>MagnetCache.kt
@Singleton
class <TrackerId>MagnetCache @Inject constructor() {
    private val byId = ConcurrentHashMap<String, String>()

    /** Records a genuinely-parsed magnet; blank inputs are
        ignored. */
```

```
    fun put(id: String, magnet: String?) {
        if (id.isBlank() || magnet.isNullOrBlank()) return
        byId[id] = magnet
    }

    /** Returns the cached magnet, or null if none was surfaced
        yet. */
    fun get(id: String): String? = byId[id]
}
```

`ConcurrentHashMap` because the search and topic features may write from different coroutines while the download feature reads. `@Singleton` so all three features share one instance per process.

Real implementations to copy:

- `core/tracker/kinozal/src/main/kotlin/lava/tracker/kinozal/magnet/KinozalMagnetCache.kt`
- `core/tracker/nnmclub/src/main/kotlin/lava/tracker/nnmclub/http/NnmclubMagnetCache.kt`
- `core/tracker/rutor/src/main/kotlin/lava/tracker/rutor/magnet/RuTorMagnetCache.kt`

8.2 Populate it on the real fetch path

Every place the production parser surfaces a magnet, write it into the cache. Two write sites:

Topic fetch (`KinozalTopic.getTopic`, `core/tracker/kinozal/…/feature/KinozalTopic.kt`):

```
override suspend fun getTopic(id: String): TopicDetail {
    val detail = parser.parse(body, topicIdHint = id)
    detail.torrent.magnetUri?.let { magnetCache.put(id,
        it) }   // ← write
    return detail
}
```

Search rows (`NnmclubSearch.search`, `core/tracker/nnmclub/…/feature/NnmclubSearch.kt`):

```
override suspend fun search(request: SearchRequest, page: Int):
        SearchResult {
    val result = parser.parse(body, pageHint = page)
    result.items.forEach { magnetCache.put(it.torrentId,
        it.magnetUri) }   // ← write
    return result
}
```

8.3 Read it from the synchronous method

```
// <TrackerId>Download.kt
class <TrackerId>Download @Inject constructor(
```

```
    private val http: <TrackerId>HttpClient,
    private val magnetCache: <TrackerId>MagnetCache,
) : DownloadableTracker {
    override suspend fun downloadTorrentFile(id: String):
        ByteArray =
        http.download("$baseUrl/download.php?id=$id")

    override fun getMagnetLink(id: String): String? =
        magnetCache.get(id)   // ← read
}
```

8.4 Share ONE instance across the clone path

When a provider is cloned (SP-4 Phase F.2), `TrackerClientFactory.create` builds per-call feature instances routed at the clone's `primaryUrl`. The magnet cache MUST be the **shared singleton** so a topic view feeds the download lookup — the cache keys by topic id, which is mirror-independent. From `core/tracker/kinozal/src/main/kotlin/lava/tracker/kinozal/KinozalClientFactory.kt`:

```
override fun create(config: PluginConfig): TrackerClient {
    val override = config.cloneBaseUrlOverride
    if (override != null) {
        return KinozalClient(
            // ...
            topic    = KinozalTopic(http, topicParser,
        magnetCache, override),
            download = KinozalDownload(http, magnetCache,
        override),  // SAME magnetCache
        )
    }
    return clientProvider.get()
}
```

8.5 Why this is honest (not a bluff)

- `get(id)` returns `null` when an id was never surfaced — an honest "not synchronously available without an HTTP fetch" per the contract, **never** a fabricated string.
- The cache is only ever populated with magnets the **production parser** extracted from a real page.
- Once the user has opened a topic (or seen a search row), the synchronous lookup returns the **real** magnet — no extra HTTP round-trip, no fabrication, no hidden fetch inside the synchronous method.

The architecture-level diagram of this pattern lives in docs/architecture/tracker-sdk.md §6.

---

9. Proving a download is real — the torrent validators

`core:common` ships two validators that turn "we returned bytes / a string" into "we returned a **genuinely-usable** download". They are the canonical anti-bluff way to assert a `Downloadable` feature actually works (the Anti-Bluff Pact's "primary assertion on user-visible state" — a download a real BitTorrent client would accept).

Validator	File	Input	Output
`TorrentFileValidator`	`core/common/src/main/kotlin/lava/common/torrent/TorrentFileValidator.kt`	`.torrent ByteArray`	`DownloadValidationResul`
`MagnetLinkValidator`	`core/common/src/main/kotlin/lava/common/torrent/MagnetLinkValidator.kt`	`magnet: String`	`DownloadValidationResul`

Both produce the **same** result type, keyed on the same lowercase 40-char info-hash, so a `.torrent` file and a magnet for the same content cross-check:

```
data class DownloadValidationResult(
    val valid: Boolean,
    val infoHashHex: String?,   // lowercase 40-char hex (SHA-1),
        or null
    val reason: String?,        // human-readable reason when !
        valid
)
```

9.1 `TorrentFileValidator`

A `.torrent` is valid only when **all** hold (verbatim from the class KDoc):

- the bytes are well-formed bencode with a dictionary at the top level;
- that dictionary contains an `info` dictionary;
- `info.name` is a non-empty byte string;
- `info.piece length` is an integer strictly greater than 0;
- `info.pieces` is a byte string whose length is a **positive multiple of 20** (each piece's SHA-1 is exactly 20 bytes; empty `pieces` describes a torrent with no data and is not a real download).

The info-hash is `SHA-1(verbatim bencoded info dict)` — computed over the exact original bytes (recovered via `valueRanges`), not a re-encoding, so it matches what a real BitTorrent client and the magnet form compute. (The bencode parser is `core/common/src/main/kotlin/lava/common/torrent/Bencode.kt`.)

```
val result = TorrentFileValidator().validate(downloadedBytes)
assertTrue(result.reason ?: "", result.valid)          // primary
        user-visible assertion
assertEquals("c12fe1c0…", result.infoHashHex)           //
        identity assertion
```

9.2 `MagnetLinkValidator`

A magnet is valid when it carries `xt=urn:btih:<hash>` where `<hash>` is **40 hex chars** OR **32 base32 chars** (RFC 4648). The extracted hash is normalized to lowercase hex, so it compares directly against `TorrentFileValidator`'s output. Links with multiple `xt` entries are accepted as long as one is a valid `urn:btih`.

```
val result = MagnetLinkValidator().validate(magnetUri)
assertTrue(result.reason ?: "", result.valid)
```

9.3 Use them as the download-feature acceptance assertion

A `DownloadableTracker` real-stack test (Step 6/7 of the recipe) MUST assert on the validator output, not on "bytes were non-empty":

```
@Test fun `download yields a real torrent`() = runTest {
    val bytes = download.downloadTorrentFile(KNOWN_ID)
    val v = TorrentFileValidator().validate(bytes)
    assertTrue("torrent invalid: ${v.reason}", v.valid)
}

@Test fun `getMagnetLink yields a real magnet after a topic
        view`() = runTest {
    topic.getTopic(KNOWN_ID)                                    //
        populates the magnet cache (§8)
    val magnet = download.getMagnetLink(KNOWN_ID)!!
    val v = MagnetLinkValidator().validate(magnet)
    assertTrue("magnet invalid: ${v.reason}", v.valid)
}
```

Existing tests to mirror:

- `core/common/src/test/kotlin/lava/common/torrent/TorrentFileValidatorTest.kt`
- `core/common/src/test/kotlin/lava/common/torrent/MagnetLinkValidatorTest.kt`

> **Server-side counterpart:** `lava-api-go`'s Torznab client (`internal/jackett/`) faces the same "is this download real?" problem and handles the 302→magnet edge case so a magnet is captured rather than auto-followed. See docs/architecture/local-stack-topology.md §4.2.

Last updated: 2026-06-08 (added §8 magnet-cache pattern + §9 download validators; §1–§7 unchanged from the 2026-05-01 SP-3a polish).